ТОО ”Колледж Хекслет”

# Лабораторная работа №3

JWT-аутентификация

**Выполнено студентом:**

Дубинин Кирилл

## Практическое задание

Раздел RO4_4. JWT-аутентификация.

Цель: Реализовать выдачу JWT после успешного login, передачу access token через Authorization: Bearer и middleware requireAuth для защиты REST endpoints.

Условие: RO4_4/lesson3/ЛР-3.pdf

## Требования задания

• Установлен пакет jsonwebtoken.

• JWT secret хранится в .env.

• Login возвращает accessToken.

• Token имеет ограниченный срок действия.

• В JWT используется sub для id пользователя.

• JWT передаётся через Authorization: Bearer.

• requireAuth проверяет JWT через jwt.verify().

• При успешной проверке создаётся req.user.

• GET /api/auth/me недоступен без token.

• Неверный или expired token возвращает 401.

• passwordHash и JWT_SECRET отсутствуют в JWT payload.

## Ход выполнения

После login сервер подписывает access JWT через HS256 со сроком 15 минут, sub, issuer и audience. Случайный JWT_SECRET хранится только в .env. requireAuth извлекает Bearer token и вызывает verify() с ограничением алгоритма. /api/auth/me и /api/orders/my требуют JWT. Изменённая подпись и истёкший exp дают 401. Payload можно декодировать без секрета; доверять ему можно только после проверки подписи.

Проект: RO4_4/lesson3

Запуск: npm install; npm start. Для PostgreSQL параметры подключения задаются в .env по образцу .env.example. Порядок применения SQL и миграций приведён в README.md проекта.

## services/tokenService.js

services/tokenService.js

```
import jwt from 'jsonwebtoken';
function config() {
    const secret = process.env.JWT_SECRET;
    if (!secret || Buffer.byteLength(secret) < 32) throw new Error('JWT_SECRET must
contain at least 32 bytes');
    return { secret, issuer: process.env.JWT_ISSUER || 'express-shop', audience:
process.env.JWT_AUDIENCE || 'express-shop-api' };
}
export function createAccessToken(user) {
    const { secret, issuer, audience } = config();
    return jwt.sign({ role: user.role }, secret, { algorithm: 'HS256', subject:
String(user.id), issuer, audience, expiresIn: process.env.JWT_EXPIRES_IN || '15m' });
}
export function verifyAccessToken(token) {
    const { secret, issuer, audience } = config();
    const payload = jwt.verify(token, secret, { algorithms: ['HS256'], issuer, audience
});
    if (!payload || typeof payload !== 'object' ||
!Number.isSafeInteger(Number(payload.sub)) || Number(payload.sub) <= 0 ||
!Number.isFinite(payload.exp)) throw new Error('Invalid token claims');
    return payload;
}
```

Листинг 1. services/tokenService.js

## middleware/authMiddleware.js

middleware/authMiddleware.js

```
import { verifyAccessToken } from '../services/tokenService.js';
export function requireAuth(req, res, next) {
    const match = /^Bearer ([^\s]+)$/i.exec(req.get('authorization') || '');
    if (!match) return res.status(401).json({ error: 'Authentication required' });
    try {
        const payload = verifyAccessToken(match[1]);
        req.user = { id: Number(payload.sub), role: payload.role };
        next();
    } catch { res.status(401).json({ error: 'Invalid or expired token' }); }
}
```

Листинг 2. middleware/authMiddleware.js

## controllers/authController.js

controllers/authController.js

```
import { registerUser, authenticateUser, findUserById } from
'../services/authService.js';
import { createAccessToken } from '../services/tokenService.js';
export async function register(req, res, next) {
    try { res.status(201).json({ user: await registerUser(req.body) }); }
    catch (error) { next(error); }
}
export async function login(req, res, next) {
    try {
        const user = await authenticateUser(req.body.email, req.body.password);
        if (!user) return res.status(401).json({ error: 'Invalid email or password' });
        res.json({ user , accessToken: createAccessToken(user) });
    } catch (error) { next(error); }
}
export async function getCurrentUser(req, res, next) {
    try {
        const user = await findUserById(req.user.id);
        if (!user) return res.status(401).json({ error: 'User not found' });
        res.json({ user });
    } catch (error) { next(error); }
}
```

Листинг 3. controllers/authController.js

## routes/authRoutes.js

routes/authRoutes.js

```
import { Router } from 'express';
import { register, login, getCurrentUser } from '../controllers/authController.js';
import { validateRegister, validateLogin } from '../middleware/validateAuth.js';
import { requireAuth } from '../middleware/authMiddleware.js';
const router = Router();
router.post('/register', validateRegister, register);
router.post('/login', validateLogin, login);
router.get('/me', requireAuth, getCurrentUser);
export default router;
```

Листинг 4. routes/authRoutes.js

## routes/orderRoutes.js

routes/orderRoutes.js

```
import { Router } from 'express';
import { getOrders, getOrderById, createOrderController } from
'../controllers/orderController.js';
import { validateId } from '../middleware/validateId.js';
import { validateOrder } from '../middleware/validateOrder.js';
import { requireAuth } from '../middleware/authMiddleware.js';
import { getMyOrders } from '../controllers/orderController.js';
const router = Router();
router.get('/my', requireAuth, getMyOrders);
router.param('id', validateId);
router.get('/', getOrders);
router.get('/:id', getOrderById);
router.post('/', validateOrder, createOrderController);
export default router;
```

Листинг 5. routes/orderRoutes.js

## Проверка результата

Проверено: 05.11.2026 13:38 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Реальный PostgreSQL

• HTTP endpoints

• CRUD сохраняется в PostgreSQL

• UNIQUE и FOREIGN KEY; категории перенесены без потери товаров

• Связи 1:1, 1:N, M:N; CHECK(quantity > 0); составной ключ

• include/select, relation filter some, историческая сумма заказа

• Миграции description/slug применены; повторный seed без дублей

• Атомарный заказ COMMIT/ROLLBACK; цена из БД; валидация позиций

• Регистрация/login; одинаковая ошибка для неверных credentials

• Argon2id, разные salt, пароль и хеш отсутствуют в ответах

• JWT: подпись, sub/iat/exp, issuer/audience, Bearer; изменённый и просроченный token → 401

• Chromium: login через fetch, сообщение при неверном пароле, имя после входа, очистка поля пароля

• Prisma Studio открыт в Chromium; таблица products содержит Keyboard

• Chromium: login через fetch, сообщение при неверном пароле, имя после входа, очистка поля пароля

## HTTP-запросы

GET /api/products → 200

GET /api/products/1 → 200

GET /api/products/999999 → 404

GET /api/products/abc → 400

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?minPrice=abc → 400

GET /api/products?inStock=maybe → 400

GET /api/products?minPrice=500&maxPrice=100 → 400

POST /api/products → 403

POST /api/products → 400

POST /api/products → 201

PATCH /api/products/6 → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

POST /api/products → 409

GET /api/orders/1 → 200

GET /api/orders?productId=1 → 200

GET /api/orders/999999 → 404

POST /api/products → 201

DELETE /api/products/8 → 200

POST /api/orders → 201

POST /api/orders → 409

POST /api/orders → 400

POST /api/orders → 400

POST /api/orders → 400

POST /api/orders → 404

POST /api/auth/register → 201

POST /api/auth/register → 409

POST /api/auth/register → 400

POST /api/auth/register → 400

POST /api/auth/register → 400

POST /api/auth/login → 200

POST /api/auth/login → 401

POST /api/auth/login → 401

GET /api/auth/me → 401

GET /api/auth/me → 401

GET /api/auth/me → 200

GET /api/auth/me → 401

GET /api/orders/my → 401

GET /api/orders/my → 200

GET /api/auth/me → 401

GET /api/unknown → 404

## Примеры ответов

### POST /api/auth/register → 201


```
{
  "user": {
    "id": 4,
    "name": "Кирилл",
    "email": "kirill@example.com",
    "role": "user"
  }
}
```

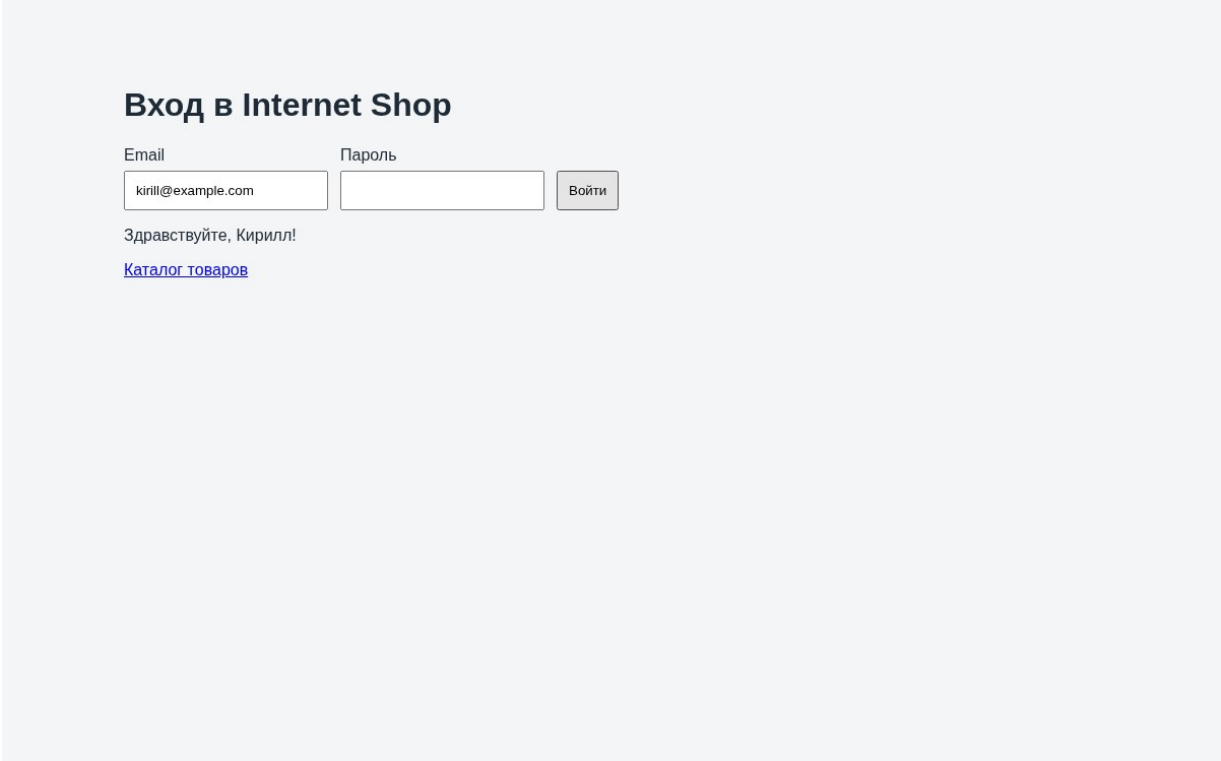

### POST /api/auth/login → 200


```
{
  "user": {
    "id": 4,
    "name": "Кирилл",
    "email": "kirill@example.com",
    "role": "user"
  },
  "accessToken": "[скрыто]"
}
```


### GET /api/auth/me → 200

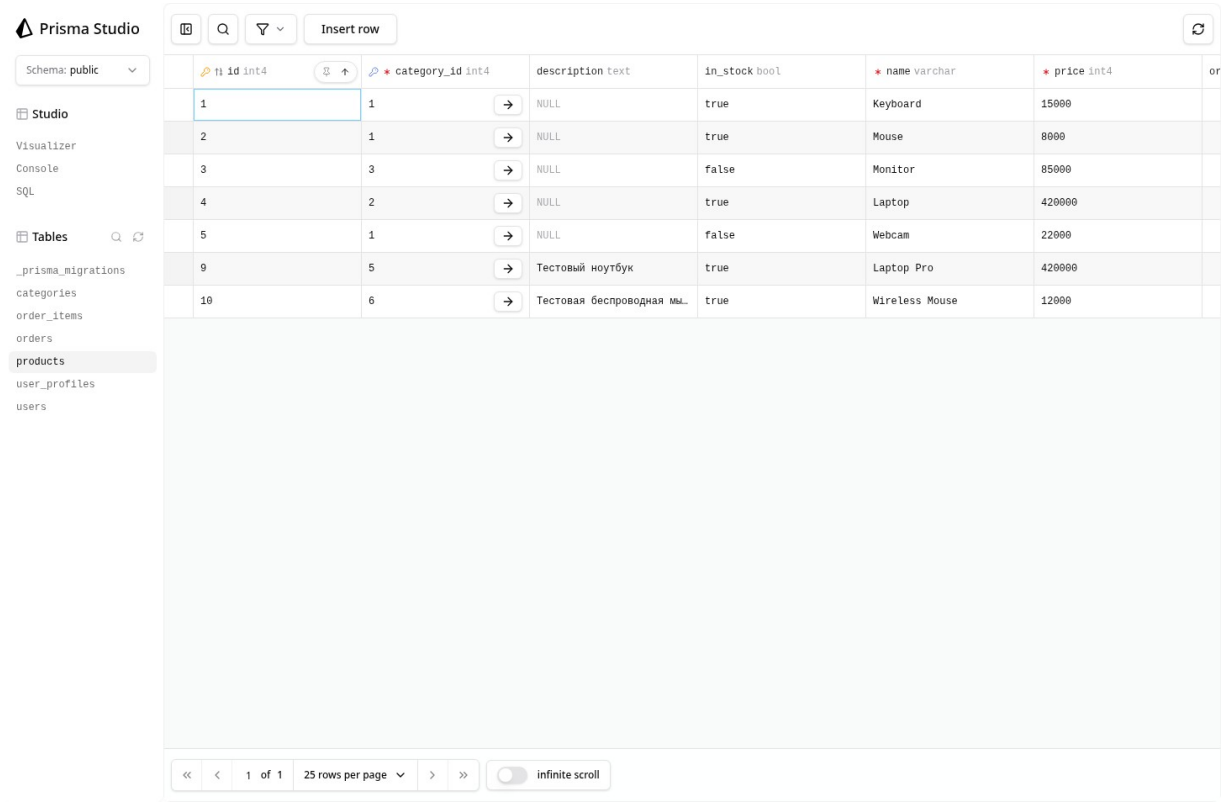

```
{
  "user": {
    "id": 4,
    "name": "Кирилл",
    "email": "kirill@example.com",
    "role": "user"
  }
}
```

# Проверка в браузере Chromium




Скриншот работающего приложения после выполнения действий через интерфейс.

## Проверка данных в Prisma Studio




Prisma Studio: таблицы и данные проверенной учебной базы.

## Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.